Assignment 3 - Classification of Image Data

Negar Akbarpouran Badr (negar.akbarpouranbadr@mail.mcgill.ca) Group **24**
Victoria Hason (victoria.hason@mail.mcgill.ca) 25 March 2025

1 Abstract

This paper aims to use multilayer perceptron (MLP) to classify image data representing ancient Japanese characters into 10 categories, a task important for the protection and transmission of Japanese history. The MLP algorithms were implemented from scratch to gain insight into their implementation. Our experiments involved analyzing accuracy over various hyperparameters, such as number of hidden units, number of layers, and activation functions. We then experimented with L2 regularization, and compared the accuracy received using our MLP to the accuracy when using a ConvNet. Our results demonstrate that the performance of the ConvNet is superior, followed closely by a 2-layer ReLU MLP with 256 hidden units and L2 regularization. We also found that the performance of an MLP increases with PCA pre-processing.

2 Introduction

Image classification is a fundamental task in the field of computer vision, with applications ranging from medical diagnosis to historical preservation. This paper focuses on using a multilayer perceptron (MLP) to classify images of ancient Japanese characters into their distinct categories. Accurate classification of these characters is significant for the protection and transmission of Japanese cultural heritage.

The dataset used in this study consists of grayscale images of ancient Japanese characters, preprocessed and normalized to ensure consistent input dimensions. Unlike Convolutional neural networks (CNNs), which are often preferred for image classification, MLPs treat input images as flattened vectors. This approach presents both challenges and opportunities in understanding how densely connected networks interpret visual data.

The primary objective of this research was to examine how different hyperparameters influence classification accuracy. Specifically, we investigated the effects of varying the number of hidden layers, the number of units per layer, and the choice of activation functions. Additionally, we applied L2 regularization to mitigate overfitting and compared our MLP's performance to that of a CNN to assess the trade-offs between the two implementations.

Key findings from our experiments include:

1. Impact of Network Depth and Width: Classification accuracy improved with both the number of hidden layers and the number of units per layer. For instance, a two-layer MLP with 256 units per layer achieved an accuracy of 89.25%, compared to a single-layer model with the same number of units, which reached 89.12%.
2. Activation Function Performance: ReLU and Leaky ReLU activation functions significantly outperformed the sigmoid function. A two-layer MLP using Leaky ReLU achieved an accuracy of 89.56%, while the sigmoid-based network was limited to 71.54%.
3. Regularization Effects: Applying L2 regularization with a lambda value of 0.001 further enhanced accuracy to 89.75%, suggesting that appropriate regularization can reduce overfitting while maintaining model complexity. This result was less than expected, but still provided a small increase.
4. Comparative analysis with a CNN revealed the expected superiority of convolutional networks for image classification (The best CNN configuration achieved an accuracy of approximately 93% on the test set) yet the MLP provided competitive results, especially with careful hyperparameter tuning. The insights gained from this study emphasize the how CNNs are better at classifying image data due to their ability to take tensor input, so they can understand spatial relation between pixels of images better.

Finally, in order to deepen our insights, we consulted existing literature to explore ways we could boost the classification accuracies for our MLP models that went above the necessary benchmarks. Our search led us to the paper titled: "Image classification of Kuzushiji ideograms with Convolutional Neural Networks", an in-depth evaluation of the performance of different algorithms for classifying images based off the Kuzushiji-MNIST dataset -(The same one we are using). According to this study, the performance of the machine learning models (for the study, KNN and MLP) improved when PCA was used as a feature extractor to feed the models. We decided to implement this for our MLP model and saw an increase in accuracy to 90.5% accuracy with one layer, and 91.6% with two layers, indicating the positive impact of this pre-processing step.

3 Datasets

The dataset used in this assignment is Kuzushiji-MNIST (KMNIST), a smaller portion of the original MNIST dataset, designed to promote research on historical Japanese character recognition. It consists of grayscale images of size 28×28 pixels, each representing a character from 10 different Hiragana classes. The full dataset is available on Kaggle and totals around 599 MB when compressed. For this assignment, we worked with a smaller, preprocessed subset provided via to us, containing 70,000 images.

We loaded the dataset using NumPy, obtaining 60,000 training images and 10,000 test images. The training set was then further split into 48,000 training examples and 12,000 validation examples using an 80-20 split. Each image was originally a 28×28 array of pixel values ranging from 0 to 255. For input into a Multi-Layer Perceptron (MLP), we flattened each image into a 784-dimensional feature vector.

By performing a small exploratory data analysis we confirmed that there are no missing values, and all label values are integers in the range 0 to 9, each corresponding to one of the 10 character classes. The class distribution appears to be approximately uniform, meaning the dataset is balanced, which is desirable for training classification models without introducing bias. Additionally, all pixel values were standardized to improve convergence during training.

This preprocessing pipeline ensured that the dataset was clean, well-formatted, and ready to be used by both traditional MLPs and convolutional architectures later in the assignment.

4 Results

To evaluate the impact of hyperparameters, we implemented a base MLP model with tuneable parameters. In each experiment, we kept all hyperparameters constant except the one being tested. This gave us a good general overview of the effects of each hyperparameter; however, there were downsides to this approach. For example, we fixed the number of training epochs to 20 in all experiments. In some cases, the best validation accuracy occurred at epoch 20, suggesting the model could have improved further with more epochs.

Accuracy was tracked on both the training and validation sets, and the test accuracy was reported using the epoch that achieved the best validation performance. This approach helps reduce overfitting by avoiding selection bias based on test performance.

All experiments used the same `MLP` class implementation, which included optional L2 regularization, variable activation functions, and customizable layer configurations. The architecture of the model is shown in the simplified Python snippet below:

```
class MLP:
    def __init__(..., activation="relu", l2_lambda=0.0):
        ...  # Architecture and initialization

    def forward(self, X):
        ...  # Forward propagation logic

    def backward(...):
        ...  # Backpropagation logic

    def fit(...):
        ...  # Training with tracking and best epoch selection
```

We plotted training and validation accuracy for each experiment to visualize convergence and generalization. The test accuracy was computed using the model parameters from the epoch with the highest

validation accuracy. This approach indirectly mitigates overfitting by selecting models that generalize better to unseen data.

Task 3.1: Impact of Depth and Width

We compared three configurations:

1. MLP with **no hidden layer** (i.e., a linear model).
2. MLP with **one hidden layer** (ReLU).
3. MLP with **two hidden layers** (ReLU).

Each model was trained with {32, 64, 128, 256} hidden units. As expected, increasing depth and width improved performance:

Hidden Layers	**N/A**	**32**	**64**	**128**	**256**
0 (none)	69.29%	–	–	–	–
1 layer (ReLU)	–	82.75%	86.05%	88.12%	89.12%
2 layers (ReLU)	–	83.63%	86.80%	88.58%	89.25%

Table 1: Test accuracy for different depths and widths

Increasing the number of hidden layers or units also significantly increased training time. We also noticed that using small batch sizes (e.g., 32 or 64) with no hidden layers yielded poor accuracy. Since the model lacked non-linearity, batching introduced gradient noise without meaningful transformation. A batch size of 784 (entire image row) was used in such cases for better convergence.

Task 3.2: Comparing Activation Functions

Using two hidden layers with 256 units each, we tested ReLU, sigmoid, and leaky ReLU activation functions. The output layer remained softmax:

Activation	**Test Accuracy**
ReLU	89.54%
Sigmoid	71.54%
Leaky ReLU	89.56%

Table 2: Test accuracy using different activation functions

ReLU and leaky ReLU significantly outperformed sigmoid, likely due to better gradient flow and resistance to vanishing gradients. Leaky ReLU showed a slight edge over standard ReLU.

Task 3.3: L2 Regularization

We added L2 regularization to the 2-layer ReLU model with 256 hidden units. Using $\lambda = 0.001$, the test accuracy improved slightly to **89.75%**. While the change is modest, regularization helps prevent overfitting by penalizing large weights. The relatively small gain suggests that the model was not severely overfitting in the first place, though the benefit might be more visible with larger networks or more training epochs.

Additional Experiments

As mentioned in the introduction, we implemented PCA (Principle Component Analysis) preprocessing to attempt to improve the accuracy of our MLP. In doing so, (following along from the code from this github link: **REF**), we reduced the data set from size [N x 784] to [N x 100], keeping only the dimensions of the data that contained the most variance (essentially, extracting the most significant features). This step improves space, time, and accuracy by reducing dimensionality and noise. The results of the models where PCA was used are as follows:
Below are plots for our best performing MLP model's accuracies per epoch.

Hidden Layers	N/A	32	64	128	256
1 layer (ReLU)	–	83.32%	87.06%	89.37%	90.5%
2 layers (ReLU)	–	85.21%	86.60%	89.80%	91.16%

Table 3: Test accuracy for different depths and widths

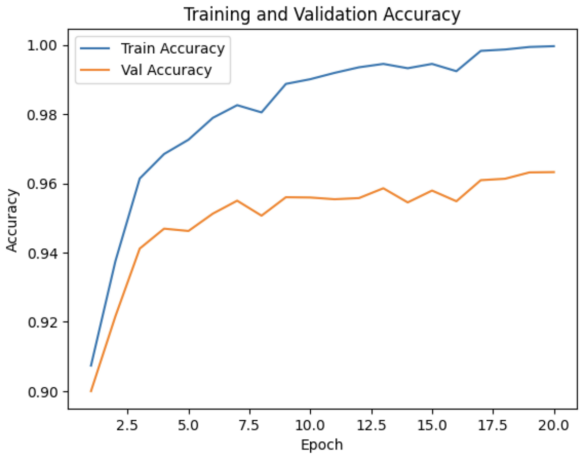

Figure 1: 2-layer, 256 hidden units, PCA

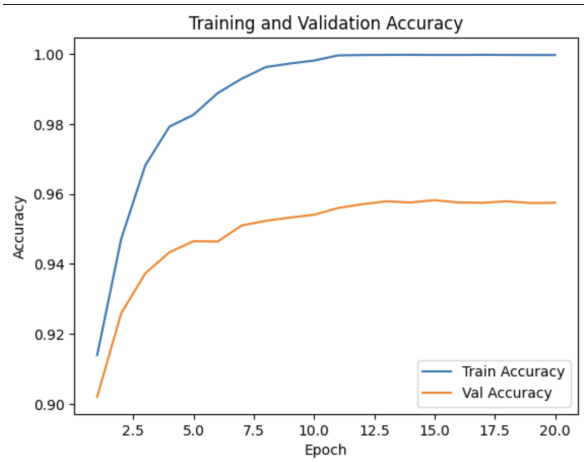

Figure 2: 2-layer, 256 hidden units, non-PCA

Task 3.4: CNN Architecture

To evaluate the impact of convolutional architectures on image classification performance, we defined a base CNN model and experimented with various hyperparameters. The model consisted of the following components:

- 3 Convolutional Layers with filter sizes of 32, 64, and 64 respectively, and kernel sizes of (3, 3). These layers are designed to detect local patterns such as edges, curves, and textures. Stacking three convolutional layers allowed the network to build increasingly abstract representations of the input data, from low-level features to higher-level shape structures.
- 3 Max Pooling Layers with a pool size of (2, 2) were interleaved with the convolutional layers to downsample the feature maps. This reduced the spatial dimensions and computational cost, while also introducing a degree of translational invariance by retaining only the most prominent feature in each 2×2 region.
- 1 Flatten Layer converted the 2D output of the convolutional layers into a 1D vector suitable for input into the fully connected layers.
- 2 Fully Connected (Dense) Layers, which learn global representations from the extracted features. We experimented with different sizes for the hidden units: 32, 64, 128, 256. As expected, increasing the number of units improved model capacity. The highest validation accuracy (93%) was achieved with 256 units, likely due to the model's increased ability to learn complex decision boundaries. However, further increases might lead to overfitting, especially with limited data.
- 2 Dropout Layers were used after the fully connected layers to reduce overfitting by randomly setting a fraction of neuron outputs to zero during training. We experimented with dropout rates in the range of 0.0 to 0.5, and found that 0.2 and 0.3 yielded the best performance. Lower dropout (e.g., 0.1) was insufficient to regularize the model, while higher rates (e.g., 0.5) overly restricted learning by discarding too much information.
- The Output Layer used softmax activation to produce probability distributions over the 10 classes in the Kuzushiji-MNIST dataset.

Overall, the CNN significantly outperformed the MLP baseline. The best CNN configuration achieved an accuracy of approximately 93% on the test set, compared to 88% for the best-performing MLP. This improvement stems from the CNN's ability to exploit the 2D spatial structure of image data using shared weights and local receptive fields — something MLPs are not designed to capture.

The training and validation accuracy curves over epochs (shown in our notebook) indicate that CNNs not only converge faster than MLPs but also generalize better with appropriate regularization and architecture choices.

5 Discussion and Conclusion

This study explored the use of MLPs for classifying ancient Japanese characters, comparing their performance to CNNs. Our results showed that MLP accuracy improved with increased depth, width, and the use of ReLU or Leaky ReLU activation functions. L2 regularization provided a modest performance boost, and PCA preprocessing further enhanced accuracy, achieving up to 91.16%. While CNNs outperformed MLPs due to their ability to capture spatial relationships, MLPs demonstrated competitive results with appropriate hyperparameter tuning. Future research could explore additional architectural variations, optimization techniques, or hybrid models to further close the performance gap.

6 Group Member Contributions

This assignment was completed by Victoria Hason, Negar Akbarpouran Badr and Melek Deniz Colak. Negar and Victoria collaborated on the writing of the report. (Victoria worked on abstract, introduction, part of results, discussion and conclusion, and Negar worked on datasets and part of results). Negar worked on implementing the main class and running CNN experiments, Victoria worked on data preprocessing, and extra reaserch and experiments. Deniz worked on performing the gradient check.

7 References

Lucimara R. Martins, Maŕio Muramatsu Junior and Adriane B. S. Serapião. *Image classification of Kuzushiji ideograms with Convolutional Neural Networks.*